

FINAL IMPLEMENTATION BASELINE

GHOST

Technical Specification v2.0

Privacy-preserving Forum & Messenger for Android

Document version: 2.0

Status: FINAL - Implementation Baseline

Date: 20 August 2026

Replaces: GHOST Technical Specification v1.0

This document defines product scope, security guarantees, normative requirements, architecture, acceptance gates and the path to production. A specification is not a substitute for an independent security audit.

Document control

Field	Value
Version	2.0
Status	FINAL - Implementation Baseline
Effective date	20 August 2026
Scope	Android client, blind relay network, cryptographic protocols, local storage, payment entitlement, infrastructure and release operations
Supersedes	GHOST Technical Specification v1.0
Normative language	MUST / MUST NOT = mandatory; SHOULD / SHOULD NOT = recommended unless an ADR approves an exception; MAY = optional
Review cadence	Quarterly, and immediately after any protocol, cryptographic, payment or distribution change

Major corrections from v1.0

The final baseline corrects the 256-bit BIP39 backup to 24 words; separates identity, messaging and wallet keys; removes custom Double Ratchet and simplified MLS; reconciles 256 KB media chunks with 64 KB relay blobs; defines scoped privacy guarantees; replaces the incomplete ZK sketch with a verifiable entitlement model; promotes secure updates and onboarding validation to launch blockers; and replaces the 16-week launch claim with gated delivery.

Contents

Document control	2
Contents	2
1. Executive summary	5
1.1 Product outcomes	5
1.2 Non-goals	5
2. Architectural principles and privacy guarantees	6
2.1 Guarantee matrix	6
3. Terminology, scope and release policy	7
3.1 Release stages	7
4. Functional requirements	8
4.1 Identity and onboarding	8
4.2 Cryptography and messaging envelope	8
4.3 Forum channels	9
4.4 Direct messages	9
4.5 Blind relay network	9
4.6 Entitlement, blockchain and referral	10
4.7 Android application	10

4.8 Operations and distribution	11
5. Non-functional requirements	12
6. System architecture	13
6.1 Android client modules	13
6.2 Relay components	13
6.3 Control plane	14
6.4 Trust boundaries	14
7. Cryptographic protocol requirements	15
7.1 Key derivation and lifecycle	15
7.2 Direct-message protocol	15
7.3 MLS forum protocol	15
7.4 Media encryption and fragmentation	16
7.5 Post-quantum policy	16
8. Data model and storage	17
8.1 Storage rules	17
9. Protocol and API contracts	18
9.1 Relay API	18
9.2 Client to entitlement contracts	18
10. Entitlement, payments and zero-knowledge design	19
10.1 Entitlement statement	19
10.2 Payment-rail rollout	19
10.3 Treasury and referral controls	19
11. Security, privacy and abuse controls	20
11.1 Logging and telemetry policy	20
11.2 Abuse resistance	20
12. Deployment, distribution and operations	21
12.1 Environments	21
12.2 Secure update mechanism	21
12.3 Operational runbooks	21
13. Verification and testing strategy	22
13.1 Production security gate	22
14. Sequential implementation roadmap	23
14.1 Minimum delivery team	23
14.2 Definition of Done	24

15. Risk register	25
Appendix A. Technology baseline and version policy	26
Appendix B. Repository structure	27
Appendix C. Production acceptance checklist	28
Appendix D. Normative references	29
Appendix E. Change log	30

1. Executive summary

GHOST is an invite-only Android forum and messenger designed so that service operators do not possess content-decryption keys. Direct messages use a maintained Signal protocol implementation; forum channels use MLS compliant with RFC 9420; local state is encrypted; relay nodes store and forward ciphertext; blockchain components hold commitments and control-plane state, never message content.

The product goal is privacy by minimization and cryptographic separation, not an absolute promise of anonymity. GHOST protects content confidentiality against relays, storage providers and the application operator when endpoints are uncompromised. It reduces metadata exposure through onion routing, fixed packet buckets, short-lived capabilities and aggregated operations. It cannot protect a compromised device, guarantee anonymity against a global passive adversary, or hide facts already visible to an external payment rail.

Launch principle

No feature may be marketed as secure, private or zero-knowledge until its threat-model claim has an automated acceptance test or an independent review. The production release requires all P0 requirements, measured NFRs and closure of external audit findings.

1.1 Product outcomes

- Private 1-to-1 messaging with forward secrecy and post-compromise recovery provided by maintained protocol libraries.
- Encrypted forum channels with authenticated membership, epoch rotation and local moderation.
- Invite-only onboarding with pseudonymous identity, secure recovery and no email, phone number or KYC collected by GHOST.
- Blind content-addressed relay storage with capability-based access, TTL and multi-jurisdiction redundancy.
- Subscription entitlement that is not linkable to the GHOST public identity by the application operator under the defined payment model.
- Signed, anti-rollback updates for an application distributed outside Play Store.

1.2 Non-goals

- Protection when the Android endpoint, keyboard, screen or operating system is compromised.
- Absolute anonymity against an adversary observing the entire network and all payment rails.
- Central content moderation or server-side plaintext search.
- Recovery of a lost identity when the user has neither the mnemonic nor an enrolled recovery mechanism.
- Hiding a cryptocurrency transaction from the cryptocurrency network itself.
- A custom cryptographic protocol designed by the application team.

2. Architectural principles and privacy guarantees

ID	Principle	Normative rule
CP-01	Endpoint key ownership	Only user endpoints possess content-decryption keys. Operator infrastructure MUST NOT receive them.
CP-02	Ciphertext outside endpoints	Message and media plaintext MUST NOT cross the endpoint trust boundary.
CP-03	Pseudonymous identity	GHOST MUST NOT require email, phone, legal name or KYC. Payment rails may impose independent requirements.
CP-04	Minimal blockchain state	Blockchain MUST contain commitments, entitlement state and release state only. No message, media, channel membership or social graph.
CP-05	Unlinkable entitlement	Proof of active entitlement MUST NOT expose the GHOST identity or recovery seed. Payment-rail observability is documented separately.
CP-06	Blind relays	Relays MUST treat payloads as opaque blobs and MUST NOT log source IPs by default.
CP-07	Metadata resistance	Production traffic MUST use multi-hop routing and packet-size buckets. The guarantee is resistance, not elimination.
CP-08	Authenticated distribution	Every production update MUST be signed, hash-verified and protected against rollback.
CP-09	No custom crypto	Production protocols MUST use maintained, reviewed libraries and standard constructions. Exceptions require an ADR and external review.
CP-10	Verifiable claims	Security and performance claims MUST be traceable to tests, measurements or audit evidence.

2.1 Guarantee matrix

Asset	Protection	Who can access	Explicit limitation
Message content	E2E encrypted	Endpoints participating in the session/group	Compromised endpoint, malicious recipient
Local history	SQLCipher plus Keystore-wrapped DB key	Unlocked application on the endpoint	Unlocked/root-compromised device
Relay payload	Opaque ciphertext blob	Client holding capability and content key	Traffic size/timing leakage within stated limits
Payment entitlement	Proof/commitment separated from GHOST ID	User and payment rail according to rail policy	Blockchain/payment rail can observe its own transaction
Social graph	Not stored centrally as plaintext	Endpoints and group participants	Timing correlation and malicious group members
Software update	Developer signature plus manifest/hash validation	Release operator controls signing policy	Signing-key compromise until emergency rotation

3. Terminology, scope and release policy

Term	Definition
Identity	Pseudonymous public identifier derived from the Ed25519 public key hash.
Device	One Android installation with device-specific cryptographic state.
DM	A 1-to-1 encrypted session using libsignal semantics.
Channel	An MLS group used as an encrypted forum thread.
Blob	Opaque relay object with maximum serialized size of 65,536 bytes.
Capability	Unforgeable authorization token granting scoped access to one or more blobs.
Entitlement	Time-bounded right to use paid features, proven without revealing the GHOST identity.
Operator	Entity operating release, relay or payment infrastructure. Roles SHOULD be separated.
P0	Required for first production release. Missing P0 means No-Go.
P1	Required before general availability or a stated security claim; may be disabled in a labelled private alpha.
P2	Post-launch enhancement that MUST NOT be implied by current marketing.

3.1 Release stages

Stage	Entry condition
Developer preview	Local/test infrastructure; no privacy claim; synthetic payments.
Private alpha	Real E2E protocols, staging relays, testnet entitlement, invited testers, explicit limitations.
Controlled beta	Production-like relays, audited RC candidate, limited treasury, measured SLOs.
Production	All P0 complete, external audit retested, incident response active, signed release rollout.

4. Functional requirements

All requirements in this section are normative. Acceptance evidence **MUST** be retained in CI, test reports, audit reports or release records.

4.1 Identity and onboarding

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-1.1	On first launch, the client MUST generate 256 bits of entropy using the platform CSPRNG.	P0	Two fresh installations produce distinct entropy; no entropy appears in logs or backups.
FR-1.2	Identity, messaging, wallet and backup-wrapping material MUST be derived with HKDF and distinct domain labels.	P0	Test vectors prove deterministic recovery and separation between branches.
FR-1.3	The public identity MUST be a versioned z-base-32 encoding of a cryptographic hash of the Ed25519 public key, prefixed ghost1.	P0	Parser rejects checksum, version and length errors; round-trip vectors pass.
FR-1.4	The 256-bit entropy backup MUST be represented by a 24-word BIP39 mnemonic. The UI MUST verify a randomized subset before completion.	P0	Reinstall plus mnemonic restores the same identity within NFR-10.
FR-1.5	Onboarding MUST require a signed invite or an explicitly configured genesis policy.	P0	No valid invite means no account activation outside genesis mode.
FR-1.6	Invite payload MUST be versioned and include inviter public key, referral commitment, nonce, expiry and signature; it MUST support deep link and QR.	P0	Tampered, expired and replayed invites are rejected.
FR-1.7	Before activation, the client MUST validate inviter eligibility and invite policy without learning or transmitting a real-world identity.	P0	Inactive inviter and revoked invite fail closed.
FR-1.8	Seed and long-lived local secrets MUST be encrypted with a Keystore-protected wrapping key; biometric/PIN policy MUST be configurable.	P0	DB/seed files are unusable outside the authorized application context.
FR-1.9	The client MUST support device revocation and identity-key change warnings.	P1	A revoked device cannot receive future group/session state.

4.2 Cryptography and messaging envelope

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-2.1	DM MUST use the maintained official libsignal Android artifacts. A custom Double Ratchet implementation is prohibited.	P0	Official session interoperability tests and two-device E2E tests pass.
FR-2.2	Forum channels MUST use an MLS implementation conforming to RFC 9420; simplified group-key encryption is prohibited in production.	P0	Welcome, add, remove, update, epoch and resync tests pass.
FR-2.3	Application payload encryption MUST provide authenticated encryption and unique nonces/keys according to the selected protocol library.	P0	Nonce-reuse property tests and tamper rejection pass.
FR-2.4	Media MUST use a random 256-bit file key and authenticated per-chunk encryption. The file key MUST be wrapped by the DM or MLS context.	P0	Relay data and manifest do not contain the plaintext key; modified chunk fails authentication.
FR-2.5	Network packets MUST be padded to 1, 4, 16 or 64 KiB buckets after serialization and before onion wrapping.	P1	Capture test observes only allowed outer sizes.
FR-2.6	Dummy traffic MAY be enabled using a battery-aware randomized policy; the UI MUST disclose its battery/network cost.	P2	Feature is off/on configurable and measured independently.
FR-2.7	Production traffic MUST traverse 2-3 relay hops with a fresh circuit policy and independent hop encryption.	P0	No single relay log/test fixture contains both source and final blob destination.
FR-2.8	Protocol messages MUST include versioning, replay protection, bounded parsing and authenticated context binding.	P0	Fuzzing and replay tests pass; unknown incompatible version fails closed.

4.3 Forum channels

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-3.1	The forum MUST be organized as invite-scoped encrypted channels. Public discovery, if introduced, MUST reveal only explicitly public metadata.	P0	Unauthorized client cannot enumerate private channel membership or content.
FR-3.2	Each channel MUST map to an MLS group with persisted epoch and authenticated membership state.	P0	Restart preserves epoch; stale epoch is detected and resynchronized.
FR-3.3	Posts MUST be encrypted and signed/authenticated before entering the relay queue.	P0	Relay capture contains no plaintext post or author signing key.
FR-3.4	Posts MUST support text, media up to 50 MiB and an authenticated client timestamp.	P0	Boundary-size, invalid-media and timestamp-signature tests pass.
FR-3.5	History MUST synchronize incrementally using opaque cursors, deduplication and idempotent writes.	P0	Offline/reconnect and crash-recovery tests produce no duplicates or loss.
FR-3.6	Filtering, mute, block and hide operations MUST be local by default.	P1	Filter state never leaves device unless explicitly exported encrypted.
FR-3.7	Channel creator or delegated role MAY pin posts using an authenticated MLS application action.	P2	Unauthorized pin is rejected; role change is auditable within the group.
FR-3.8	Member removal MUST prevent access to future epochs; history access for new members MUST follow an explicit channel policy.	P0	Removed member fails future decrypt; new-member history behavior matches policy.

4.4 Direct messages

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-4.1	DM sessions MUST support prekey establishment, asynchronous delivery, out-of-order messages and session rotation.	P0	Two-device suite covers offline recipient, reordering, replay and restart.
FR-4.2	Typing indicators MUST be encrypted, optional and disabled by default in high-privacy mode.	P2	Network observer cannot distinguish content; user control persists.
FR-4.3	DM MUST support text, media, reactions, reply references and delivery status without server-side plaintext.	P0	E2E tests pass for every message type.
FR-4.4	Disappearing messages MUST support at least 1h, 24h and 7d. Expiry is enforced locally and communicated cryptographically.	P1	Expired content and derived cache are removed on all participating test devices.
FR-4.5	DM history and session state MUST be stored only in encrypted local storage.	P0	Filesystem inspection reveals no plaintext message/session state.
FR-4.6	Users MUST be warned when a contact identity key changes and MUST be able to verify a safety code.	P0	Identity replacement creates a blocking or explicit confirmation state.
FR-4.7	Block and delete operations MUST be local, deterministic and documented regarding remote copies.	P1	UI accurately states what can and cannot be deleted remotely.

4.5 Blind relay network

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-5.1	Relay API MUST accept only bounded opaque blobs with maximum serialized size 65,536 bytes.	P0	Oversize and malformed input fail before storage allocation escalation.
FR-5.2	Blob key MUST be SHA-256 of the exact ciphertext blob; relay MUST recompute and compare on store.	P0	Hash mismatch is rejected; duplicate blob is idempotent.
FR-5.3	Store/get operations MUST require scoped, unforgeable capabilities.	P0	Missing, modified, expired or wrong-scope capabilities are rejected.
FR-5.4	Relay MUST NOT log source IP, capability, payload or stable client identifier by default.	P0	Automated log-scrub test and operator review pass.
FR-5.5	Default TTL MUST be 90 days, with bounded policy values and a documented distinction between relay retention and local history.	P1	Prune test removes expired blobs and preserves unexpired blobs.
FR-5.6	Relays MUST synchronize encrypted inventory/data through authenticated, rate-limited gossip.	P1	Three-node convergence and partition recovery tests pass.
FR-5.7	Production MUST operate at least three bootstrap relays in separate failure domains and, where feasible, jurisdictions.	P0	Failover test removes one relay without loss of service target.
FR-5.8	Transport MUST use Noise or a reviewed equivalent with relay authentication, key rotation and replay protection.	P0	MITM and stale-key tests fail closed.
FR-5.9	Abuse controls MUST use quotas, capability scopes and optional proof-of-work without introducing a stable real-world identity.	P1	Load/abuse suite prevents unbounded CPU, memory and disk use.

4.6 Entitlement, blockchain and referral

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-6.1	IdentityAnchor, if retained, MUST store only a versioned public-key commitment and status; it MUST NOT store channel or message data.	P1	On-chain schema review confirms absence of content/social graph.
FR-6.2	Subscription contract MUST verify proof of an active entitlement commitment and prevent proof replay using a nullifier.	P0	Invalid, expired and replayed proofs fail in contract tests.
FR-6.3	Referral accounting MUST allocate 10% according to a referral commitment without exposing the GHOST identity.	P0	Self-referral, cycles and duplicate payout tests pass.
FR-6.4	ContentNotary is optional and MUST remain disabled unless a privacy review proves that timing and linkage leakage are acceptable.	P2	Feature flag defaults off; no production dependency.
FR-6.5	Entitlement proof MUST constrain amount/policy, validity interval, currency/rail, commitment membership and unique nullifier.	P0	Circuit negative tests fail independently for every constraint.
FR-6.6	USDC on Base is the initial production rail. Lightning and XMR MAY be enabled only after an unlinkable credential issuance design and security review.	P0/P1	USDC P0 flow passes; each additional rail has its own approved threat model and tests.
FR-6.7	Treasury operations MUST use multisig, least privilege, rate limits and reconciled sweeps.	P0	Unauthorized signer, limit and reconciliation tests pass.
FR-6.8	The wallet branch MUST be cryptographically separate from messaging identity keys.	P0	Derivation vectors prove no key reuse and deterministic recovery.
FR-6.9	Payment rail metadata and trust assumptions MUST be disclosed to users before payment.	P0	UX/legal review confirms rail-specific disclosure.
FR-6.10	Contract upgrades, verifier-key changes and emergency pause MUST use timelock/multisig governance and publish signed notices.	P0	Upgrade and emergency drills pass on testnet.

4.7 Android application

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-7.1	UI MUST use Jetpack Compose, Material You, light/dark themes, accessibility labels and scalable text.	P0	Automated accessibility and theme tests pass on supported devices.
FR-7.2	Local relational data MUST use the current sqlcipher-android API with a Keystore-wrapped database key.	P0	Encrypted DB and migration tests pass.
FR-7.3	Background synchronization MUST use WorkManager and coroutines with idempotent queues and battery/network constraints.	P0	Doze, offline, retry and process-death tests pass.
FR-7.4	Encrypted video MUST use AndroidX Media3/ExoPlayer with local authenticated chunk decryption.	P0	Seek/resume/corruption tests pass without plaintext cache.
FR-7.5	Because distribution is outside Play Store, startup update checks, signed manifests, APK hash verification and anti-rollback are mandatory.	P0	Tampered APK/manifest and downgrade are rejected.
FR-7.6	Integrated wallet MUST derive from the wallet branch and MUST require explicit user confirmation for signing.	P0	No background signing; wrong-chain and amount warnings tested.
FR-7.7	Invite links MUST support QR encode/decode with checksum and version display.	P1	Damaged QR and unknown version fail safely.
FR-7.8	Sensitive screens MUST prevent screenshots/recents capture where supported and clear clipboard data after a short timeout.	P0	Mnemonic/seed does not appear in screenshots, recents or persistent clipboard.
FR-7.9	Backup rules MUST explicitly exclude seed, DB keys, session state and plaintext cache from cloud/device transfer unless an approved E2E export format is used.	P0	Android backup/restore inspection confirms exclusions.

4.8 Operations and distribution

ID	Normative requirement	Prio	Minimum acceptance criterion
FR-8.1	Every production artifact MUST be reproducibly built where practical, signed and accompanied by a hash and SBOM.	P0	Release pipeline verifies provenance and dependency lock.
FR-8.2	Release signing keys MUST use offline/HSM-backed storage, multi-party access and documented rotation.	P0	Key compromise and rotation drill completed.
FR-8.3	Operational telemetry MUST be aggregated and MUST exclude content, capabilities, social graph and stable user IDs.	P0	Telemetry schema passes privacy review and packet inspection.
FR-8.4	Infrastructure MUST be declarative, peer-reviewed and deployable to staging before production.	P0	Clean environment deployment and rollback pass.
FR-8.5	Incident response MUST cover relay compromise, signing-key compromise, contract exploit, data corruption and dependency vulnerability.	P0	Tabletop exercise and runbooks completed.
FR-8.6	A public security contact and coordinated vulnerability disclosure process MUST exist before production.	P0	Contact, SLA and triage workflow tested.
FR-8.7	Marketing and in-app claims MUST match the guarantee matrix and current feature flags.	P0	Release review contains signed claim checklist.
FR-8.8	Production rollout MUST support staged percentages, health gates and rollback.	P0	5/25/50/100 percent staged rollout rehearsal passes.

5. Non-functional requirements

ID	Metric	Target	Measurement condition
NFR-1	DM end-to-end latency	p95 < 2 s; p99 < 5 s	Two physical mid-range devices, healthy 3-hop route, 4 KiB payload, 30-minute run.
NFR-2	New forum post sync	p95 < 5 s	20-member MLS channel, foreground client, healthy relays.
NFR-3	Relay store throughput	>= 1,000 blobs/s/node	64 KiB mixed store/get, defined CPU/disk instance, 15-minute sustained test.
NFR-4	APK size	< 70 MiB universal; optimized split target < 50 MiB	Signed release artifact; native library impact reported separately.
NFR-5	Background battery	< 2%/hour active-sync profile; < 1%/hour idle	Defined device, cellular/Wi-Fi profiles, screen off, 6-hour test.
NFR-6	Relay availability	>= 99.5% per node; service target >= 99.9%	Monthly window excluding approved maintenance; synthetic probes.
NFR-7	Infrastructure cost	Budget envelope approved at 1,000 users	Real invoice plus bandwidth/storage scenario; no unsupported fixed \$100 promise.
NFR-8	Entitlement proof generation	p95 < 2 s initial launch target	Mid-range production device; exact circuit/backend version recorded.
NFR-9	On-chain proof verification	Within configured gas budget and p95 RPC < 3 s	Base testnet/mainnet-like test; gas and RPC measured separately.
NFR-10	Seed recovery	< 3 min median, < 5 min p95	User study includes mnemonic entry, validation and first local unlock.
NFR-11	Crash-free sessions	>= 99.5% beta; >= 99.8% production	Privacy-preserving crash telemetry, rolling 7-day window.
NFR-12	Data durability	No acknowledged message loss in fault suite	Process kill, disk full, network partition and relay restart scenarios.
NFR-13	Security findings	0 open Critical/High at production	External audit retest and dependency scan on release commit.
NFR-14	Supported scale	Documented channel/member and concurrent-user limits	Load test defines safe operating envelope; UI enforces limits.

Measurement rule

An NFR without a recorded environment, workload, percentile and artifact version is unverified. Targets may be revised through an ADR before production, but they may not be silently marked as met.

6. System architecture

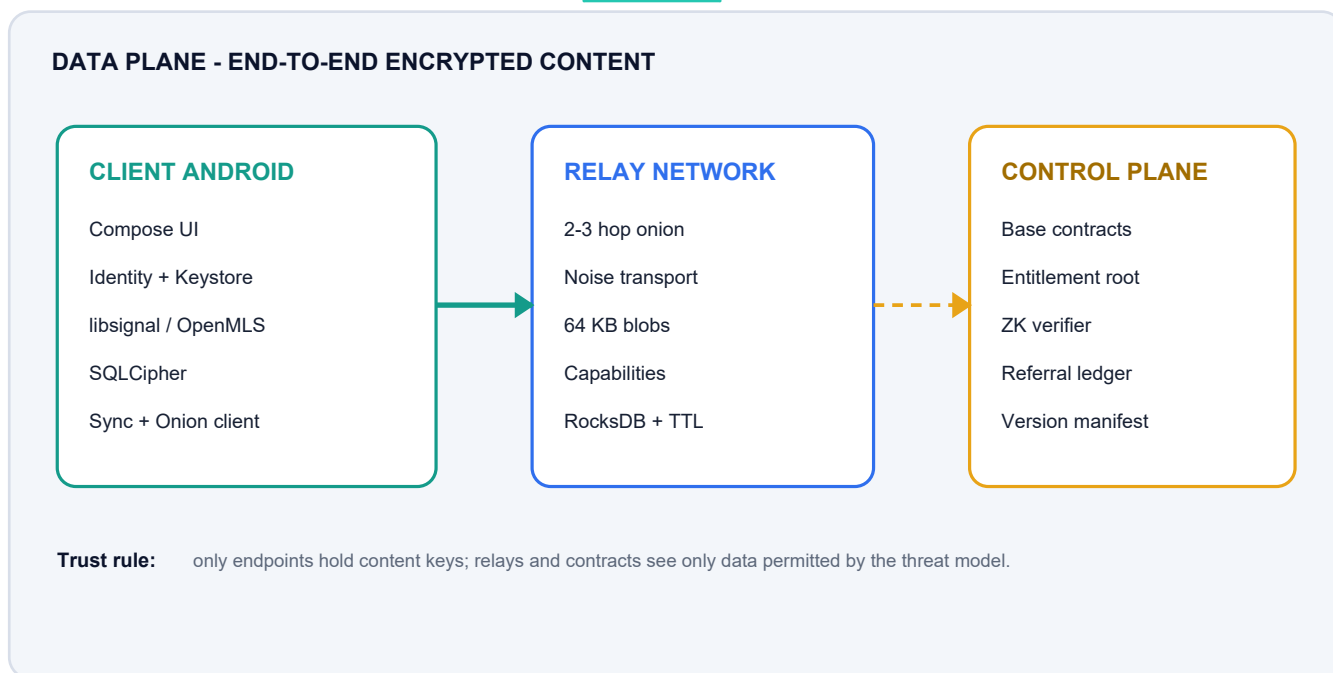


Figure 1. High-level trust and data-flow separation.

6.1 Android client modules

Module	Responsibility
identity	Seed generation, derivation, identity encoding, invite validation and recovery.
crypto	Thin wrappers over maintained libsignal/OpenMLS and platform cryptography. No protocol logic in UI.
messaging	DM and MLS orchestration, message envelope, replay/idempotency and safety state.
storage	SQLCipher schema, migrations, repositories, queue and encrypted cache.
sync	WorkManager jobs, outbox/inbox, relay selection, incremental cursors and retries.
network	Noise transport, onion circuit construction, padding and capability client.
wallet	Separated derivation branch, Base transaction signing and rail adapters.
entitlement	Proof generation, nullifier state and subscription status.
ui	Compose screens, accessibility, privacy controls and accurate security disclosures.

6.2 Relay components

- API: bounded store/get/check operations with explicit protocol versioning.
- Transport: authenticated Noise sessions, key rotation and resource limits.
- Storage: RocksDB content-addressed blobs with TTL and disk quotas.
- Capability verifier: scoped authorization without user identity.
- Gossip: authenticated inventory and data convergence across peers.
- Prune: deterministic removal of expired data and tombstone convergence where needed.
- Metrics: aggregate health only, with privacy schema enforced in code review and tests.

6.3 Control plane

The control plane consists of Base contracts, entitlement roots/verifiers, referral accounting, release manifests and operational configuration. It **MUST** remain logically separate from message delivery. Loss of the control plane may block new subscription activation but **MUST NOT** reveal or decrypt previously delivered content.

6.4 Trust boundaries

ID	Boundary	Rule
TB-1	Android process / Keystore	Seed, DB key and signing operations cross only through authorized platform APIs.
TB-2	Client / relay	Only padded onion packets and scoped capabilities cross.
TB-3	Relay / relay	Authenticated gossip; no endpoint identity propagated.
TB-4	Client / blockchain RPC	Public commitments, proofs and transactions; no message/channel content.
TB-5	Payment rail / entitlement	Rail-specific receipt or commitment transformed into unlinkable entitlement evidence.
TB-6	Build / release	Signed artifact and manifest; signing credentials isolated from CI workers.

7. Cryptographic protocol requirements

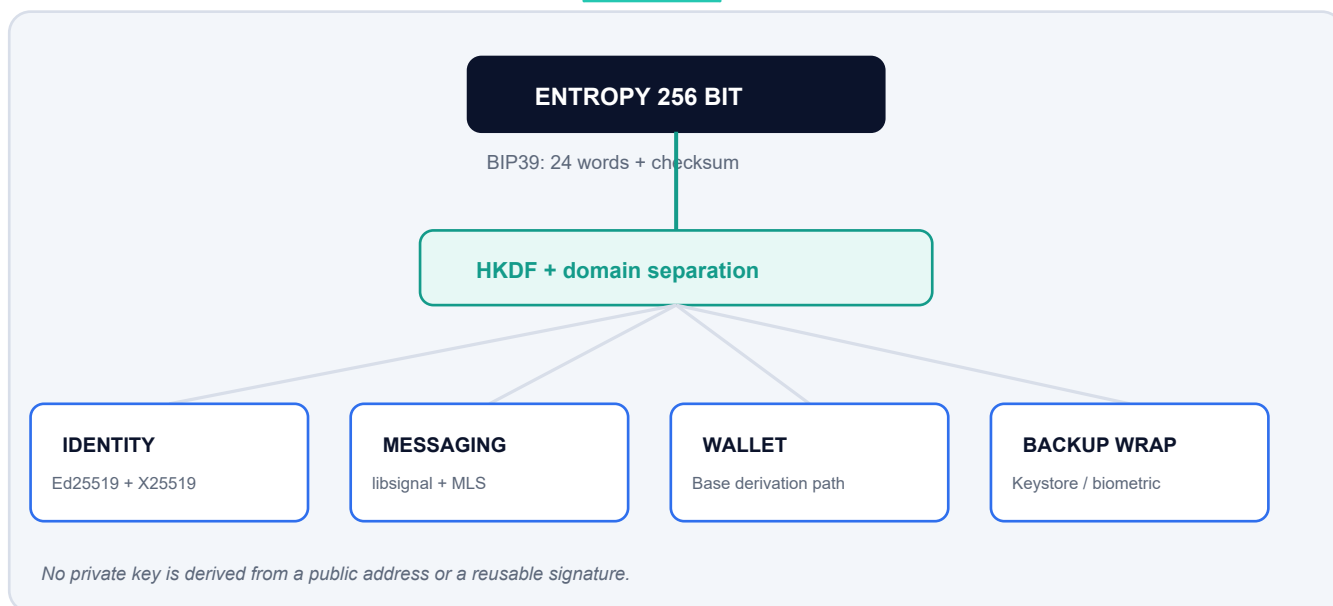


Figure 2. Key hierarchy and mandatory domain separation.

7.1 Key derivation and lifecycle

1. Generate 256 bits of entropy using Android SecureRandom backed by the platform CSPRNG.
2. Encode recovery entropy as a 24-word BIP39 mnemonic; never derive security from an address or from a reusable EIP-712 signature.
3. Derive independent branches with HKDF-SHA-256 and versioned context strings.
4. Generate or import endpoint keys using reviewed libraries; persist only encrypted key material and library session state.
5. Wrap local storage secrets using Android Keystore keys with an explicit biometric/PIN policy.
6. Rotate device/session/group keys according to protocol state; revoke devices without changing the human identity unless compromise requires it.

7.2 Direct-message protocol

DM behavior **MUST** be delegated to the official maintained libsignal artifacts selected for the Android release. The application owns identity binding, prekey publication through blind transport, session storage, UI safety state and integration tests. It **MUST NOT** reimplement X3DH/PQXDH or Double Ratchet primitives.

- Associated data binds protocol version, sender/recipient device identities and application message type.
- Session state updates and message persistence occur atomically to prevent key reuse after process death.
- Skipped-key storage is bounded to prevent memory/disk denial of service.
- Identity-key changes require a user-visible safety state before normal trust is restored.

7.3 MLS forum protocol

Channel groups **MUST** follow RFC 9420 semantics through OpenMLS or another reviewed compliant implementation. Group state includes credential binding, epoch, ratchet tree, pending proposals/commits and storage provider state.

- Welcome messages are encrypted individually and transported as opaque blobs.
- Add/remove/update operations create authenticated commits and advance the epoch.
- Old epoch secrets are removed according to the library and application history policy.
- Application messages bind channel ID, epoch, content type and client operation ID.

- State desynchronization triggers bounded resync; it must not silently fall back to a shared static key.

7.4 Media encryption and fragmentation

Step	Rule
1	Generate a random 256-bit file key and random file identifier.
2	Split plaintext into logical 256 KiB chunks for streaming and progress.
3	Encrypt each logical chunk with authenticated encryption and a nonce derived from a collision-resistant file nonce plus chunk index.
4	Fragment encrypted output and manifest into relay blobs $\leq 65,536$ bytes.
5	Hash every exact ciphertext blob; store only hash, capability and encrypted manifest reference.
6	Wrap the file key inside the DM/MLS application context and never store it beside relay ciphertext as plaintext.
7	Verify blob hash and AEAD tag before exposing decoded bytes to image/video consumers.

7.5 Post-quantum policy

ML-KEM is standardized by NIST FIPS 203, but integrating it independently into an existing messaging protocol can weaken security. GHOST MUST use the post-quantum mode exposed by the selected maintained messaging library, or complete a separate protocol design and external review. A standalone ML-KEM key pair generated only to satisfy a checklist is not sufficient.

8. Data model and storage

The schema below defines logical entities. Exact SQL belongs to versioned migrations and **MUST NOT** store raw root/session/group keys in generic application columns when the protocol library provides a dedicated secure storage representation.

Entity	Required content and constraint
identity	Public identity, encrypted seed envelope reference, derivation version, created time.
devices	Device ID, public keys, status, added/revoked times.
contacts	Local display name, identity key, trust/safety status; never real identity by default.
conversations	Contact/device references, protocol version, local state summary.
signal_state	Serialized libsignal records encrypted in SQLCipher; bounded skipped-key metadata.
messages	Operation ID, conversation/channel reference, ciphertext envelope, type, time, state and expiry.
channels	Channel ID, display metadata, local policy, current MLS epoch.
mls_state	Serialized OpenMLS group/provider state, pending commits and resync marker.
posts	Channel, encrypted envelope reference, authenticated author identity, state and media refs.
media	Encrypted manifest, blob hashes, capabilities, size/type and cache state. File key remains wrapped.
relay_queue	Idempotent operation, target policy, attempts, next retry and acknowledgement.
sync_cursor	Opaque relay/channel cursor and last successful synchronization state.
entitlement	Public commitment/root version, local proof inputs envelope, nullifier state and validity.
referral	Referral commitment and local payout view; no GHOST-to-payment-address mapping.

8.1 Storage rules

- Foreign keys and uniqueness constraints enforce conversation/channel/message integrity.
- All network-originated data is treated as untrusted and parsed with size/depth limits before DB insertion.
- Message and protocol-state mutation use transactions to avoid ratchet/epoch divergence.
- Cloud and device-transfer backup rules exclude seed, DB keys, protocol state and decrypted cache.
- Secure deletion is best effort on flash storage; the UI and documentation **MUST** not promise guaranteed physical erasure.
- Migrations are tested from every supported production schema version and have a recovery path after interruption.

9. Protocol and API contracts

9.1 Relay API

Operation	Request	Response	Mandatory validation
StoreBlob	version, blob_hash, data, capability, ttl_seconds, request_id	success, stored_hash, expiry	data <= 64 KiB; hash/capability valid; TTL bounded; idempotent request
GetBlob	version, blob_hash, capability, request_id	data, uploaded_at, expiry	capability scope valid; response bounded
CheckBlobs	version, blob_hashes[], capability	available_hashes[]	batch count bounded; no enumeration outside capability scope
GossipSync	authenticated peer stream, inventory batches	acknowledged batches	relay identity authenticated; bandwidth/depth bounded

The normative wire schema MUST live in `protocol/relay/v1/*.proto` and be generated in CI. Unknown fields follow protobuf compatibility rules; incompatible major versions fail closed.

9.2 Client to entitlement contracts

Operation	Rule
registerIdentityCommitment	Optional P1 identity commitment registration; no raw Ed25519 key.
submitEntitlementProof	Verify proof, root version, expiry policy and nullifier; record time-bounded entitlement commitment.
isEntitlementActive	Read active status by entitlement commitment, not GHOST public identity.
registerReferralCommitment	Bind entitlement flow to referral commitment under anti-self-referral policy.
claimReferral	Claim accrued referral value using authorized commitment/proof and replay protection.
publishRelease	Publish signed release-manifest hash/version through governed release registry.

10. Entitlement, payments and zero-knowledge design

Privacy claim boundary

GHOST can prevent the application identity from being linked to a payment entitlement in its own data model. It cannot make a Base, Lightning or Monero transaction invisible to that rail. Each rail requires a separate disclosure and threat model.

10.1 Entitlement statement

The production circuit MUST prove the following statement, with every predicate represented by a real constraint and negative test:

Element	Normative meaning
Private witness	Entitlement secret, payment/credential witness, amount, payment time, referral secret as applicable.
Public input	Policy/root version, subscription period, minimum amount, allowed rail/currency ID, entitlement commitment, nullifier.
Constraint 1	Witness is a member of or is authorized by the active entitlement root/issuer policy.
Constraint 2	Amount/value meets the public policy minimum and uses an allowed rail/currency.
Constraint 3	Payment/credential time is within the allowed subscription period and not expired.
Constraint 4	Entitlement commitment is derived from the witness secret and policy domain.
Constraint 5	Nullifier is derived deterministically for the entitlement/period and prevents replay.
Constraint 6	Referral commitment, when present, is valid and cannot create self-referral or payout cycles.

10.2 Payment-rail rollout

Rail	Priority	Launch condition
USDC on Base	P0	Initial production rail. Contract records/derives entitlement commitment without GHOST identity. Wallet address remains visible to Base.
Lightning BTC	P1	Enable only with blinded/unlinkable credential issuance or equivalent reviewed design. Invoice operator sees rail metadata.
Monero XMR	P1	Enable only after a verifiable entitlement issuance design, operational reconciliation and legal review.

10.3 Treasury and referral controls

- Treasury keys use multisig and role separation; CI and relay hosts hold no treasury signer.
- Sweeps are rate-limited, reconciled and observable through aggregate accounting.
- Referral payout uses commitments and batching where possible to reduce timing linkage.
- Self-referral, circular referral and duplicate entitlement claims have explicit contract invariants.
- Contract pause affects new financial actions only; it does not grant access to message content or client keys.

11. Security, privacy and abuse controls

Control family	Minimum production controls
Key management	Keystore wrapping, least use authorization, rotation, recovery and compromise playbook.
Application hardening	Release minification, dependency verification, JNI boundary checks, screenshot/clipboard controls.
Network	Noise authentication, onion routing, packet buckets, timeouts, certificate/key rotation.
Relay	No-IP logging, bounded parsing, quotas, capabilities, disk pressure and peer authentication.
Storage	SQLCipher, migration testing, backup exclusions, encrypted cache and transactional protocol state.
Blockchain	Multisig, timelock, invariant/fuzz tests, proof replay protection and limited upgrade authority.
Supply chain	Pinned dependencies, SBOM, artifact provenance, signed commits/releases and vulnerability monitoring.
Privacy claims	Claim checklist tied to active feature flags, measurements and audit evidence.

11.1 Logging and telemetry policy

Production telemetry is opt-in where required by law/policy and MUST use an allowlist schema. Free-form payload logging is prohibited.

Category	Policy
Allowed	Aggregate uptime, error class, queue depth, storage utilization, coarse latency histograms, anonymous build version.
Prohibited	Message/media bytes, keys, capabilities, seed/mnemonic, contact/channel IDs, raw IPs, payment address, stable cross-session user ID.
Retention	Shortest operational period defined by policy; automatic deletion and access audit.

11.2 Abuse resistance

- Capability-scoped quotas and rate limits instead of identity-based surveillance.
- Optional proof-of-work for unauthenticated bootstrap operations.
- Maximum blob, batch, nesting, group-size and pending-message limits.
- Local block/mute/filter controls and signed community lists as an optional user choice.
- No central plaintext scanning. Legal and product documentation must reflect this limitation.

12. Deployment, distribution and operations

12.1 Environments

Environment	Isolation rule
Development	Local/synthetic data, disposable keys, mock/test payment adapter only.
Staging	Three relays, Base testnet, production-like monitoring, separate signing and secrets.
Production	Separate accounts/failure domains, hardened images, governed contracts, audited release artifact.

12.2 Secure update mechanism

1. The client fetches a small release manifest from at least two independent locations.
2. It verifies the manifest signature against a pinned release key and validates the governed release-registry commitment when available.
3. It rejects versions lower than the installed anti-rollback floor unless an emergency recovery policy explicitly authorizes them.
4. It downloads the APK from HTTPS/IPFS sources, verifies exact hash and Android signing identity, then invokes the platform install flow.
5. Release rollout is staged and can be halted without invalidating previously installed secure versions.

12.3 Operational runbooks

- Relay compromise/isolation and key rotation.
- Release signing-key compromise and emergency trust-root rotation.
- Contract exploit, pause, treasury isolation and user notification.
- Entitlement proof/verifier regression and rollback to last compatible policy.
- Database migration failure and client recovery path.
- Critical dependency/security advisory response and forced minimum version.

13. Verification and testing strategy

Layer	Scope	Evidence/tool
Unit	Key derivation wrappers, parsers, DB repositories, capability checks, contract/circuit helpers	JUnit, Kotlin test, cargo test, Foundry, nargo
Protocol vectors	Identity encoding, libsignal integration, MLS state, media envelopes	Published/library vectors plus project fixtures
Property/fuzz	Nonce uniqueness, serialization bounds, relay parser, JNI boundary, contract invariants	jqwik/Kotest, cargo-fuzz, Foundry fuzz/invariant
Integration	Android-relay sync, relay gossip, entitlement proof and contract	Hermetic Docker/testnet harness
E2E	Invite -> identity -> entitlement -> DM/channel/media -> update	Physical devices plus emulator matrix
Load/chaos	Relay throughput, partitions, disk pressure, process death, RPC failure	k6/custom Rust bench/fault injection
Security	Mobile, crypto integration, relay, contracts, circuit, supply chain	Independent audit and retest
Usability	Mnemonic, invite, payment disclosure, recovery and safety warnings	Moderated test with representative users

13.1 Production security gate

No-Go conditions

Production is blocked by any open Critical/High security finding, unverified P0 requirement, missing release rollback, unresolved protocol-state corruption, payment/referral reconciliation failure, or marketing claim that exceeds the guarantee matrix.

14. Sequential implementation roadmap

The plan is gate-based. Tasks within a phase may run in parallel, but the exit gate is a dependency for downstream security claims. With a 5-7 person experienced team, controlled overlap yields an estimated 10-12 month calendar. A strictly sequential single-team execution is materially longer.

Phase	Outcome	Estimate	Exit gate
0	Specification v2 baseline	1-2 w	Approved scope, ADR backlog, FR/NFR acceptance matrix; Gate: contradictions closed.
1	Repository and build foundation	1-2 w	Canonical monorepo, Android/Rust/contracts/circuits skeleton, reproducible CI; Gate: clean clone builds all artifacts.
2	Threat model and security architecture	2-3 w	Assets, adversaries, trust boundaries, key lifecycle, metadata matrix; Gate: independent design review.
3	Identity and onboarding	3-5 w	256-bit entropy, 24 words, key separation, Keystore, invites, recovery; Gate: deterministic recovery and tamper/replay tests.
4	SQLCipher data layer	2-4 w	Schema, migrations, repositories, backup exclusions; Gate: encrypted persistence and crash-safe migration.
5	Blind relay v1	4-6 w	Rust store/get/check, capabilities, Noise, TTL, Docker; Gate: two-node fault and abuse suite.
6	Android sync engine	3-5 w	Outbox/inbox, WorkManager, cursors, retry/idempotency; Gate: offline/reconnect without loss/duplicates.
7	DM with libsignal	5-8 w	Prekeys, sessions, identity safety, message types; Gate: two physical devices, reordering/restart/replay suite.
8	Forum with MLS	6-10 w	OpenMLS bridge, channel lifecycle, epoch/history policy; Gate: 20-client add/remove/resync test.
9	Encrypted media	3-5 w	Chunk encryption, 64 KiB fragmentation, resume, Media3; Gate: corruption/resume/no-plaintext-cache tests.
10	Multi-relay metadata protection	5-8 w	Three relays, gossip, onion circuits, padding, quotas; Gate: failover and metadata-capture review.
11	Wallet and contracts	5-8 w	Separated wallet branch, testnet contracts, governance/invariants; Gate: audited testnet flow and upgrade drill.
12	ZK entitlement and payments	6-10 w	Constrained circuit, USDC/Base, nullifiers, referral accounting; Gate: full negative circuit/contract suite.
13	Production UX	4-6 w	Compose screens, accessibility, disclosures and error/offline states; Gate: usability completion of P0 journeys.
14	Signed distribution	3-5 w	Offline/HSM signing, manifest, hash, anti-rollback, IPFS/HTTPS; Gate: tamper/rollback/rotation drill.
15	Hardening and external audit	6-10 w	NFR measurement, fuzz/load/chaos, audit and retest; Gate: zero open Critical/High.
16	Private alpha	3-5 w	20-50 testers, staging relays/testnet, incident drills; Gate: two stable weeks and verified recovery.
17	Controlled beta	4-8 w	200-500 users, limited treasury, production-like SLOs; Gate: 30 stable days and reconciled finance.
18	Release candidate and launch	1-2 w	Go/No-Go, 5/25/50/100 rollout, rollback; Gate: signed production approval.

14.1 Minimum delivery team

Role	Accountability
Technical/security lead	Architecture, threat model, protocol integration, reviews and production Go/No-Go.
2 Android engineers	Compose, local storage, identity, sync, protocol UX and device test matrix.
Rust/network engineer	Relay, Noise, storage, gossip, onion routing and performance.
Web3/ZK engineer	Contracts, circuit, wallet/rail integration, governance and financial invariants.
QA automation	Requirement traceability, device E2E, fault/load suites and release evidence.
SRE/DevOps	Infrastructure, release pipeline, observability, incident and staged rollout.
External auditor/legal	Independent security assessment and payment/privacy/distribution review.

14.2 Definition of Done

- Acceptance criteria pass and evidence is linked to the requirement ID.
- Unit and relevant integration/E2E tests pass in CI.
- Error, retry, migration and rollback behavior is implemented and tested.
- No secret or prohibited metadata is added to logs/telemetry/backups.
- Security review is complete for trust-boundary or cryptographic changes.
- Documentation, protocol schema and operational runbook are updated.
- No placeholder, simulated success or unresolved TODO remains on the production path.

15. Risk register

ID	Risk	Prob	Impact	Mitigation
R-01	Seed loss	Med	Critical	Mandatory verified backup, clear recovery UX, optional reviewed device-transfer mechanism.
R-02	Endpoint compromise	Med	Critical	Keystore, screen/clipboard protection, dependency hardening, clear limitation disclosure.
R-03	Protocol integration bug	Med	Critical	No custom crypto, vectors, atomic state, external audit, staged rollout.
R-04	Relay correlation	Med	High	2-3 hops, padding, circuit rotation, separated operators, metadata testing.
R-05	Relay seizure/outage	Low/Med	High	Three failure domains, encrypted blobs, gossip, failover drills.
R-06	Contract/verifier exploit	Low/Med	Critical	Invariants, audit/retest, multisig/timelock, limited pause and treasury caps.
R-07	Payment identity linkage	Med	High	Entitlement separation, rail-specific threat model/disclosure, batching/blinded issuance for added rails.
R-08	ZK performance	Med	Med	Circuit profiling on target devices, policy simplification, proof caching where safe.
R-09	Supply-chain compromise	Low/Med	Critical	Pinned dependencies, SBOM, provenance, isolated signing, emergency minimum version.
R-10	Abuse/spam	High	High	Capabilities, quotas, proof-of-work, local moderation and bounded resource use.
R-11	Update signing-key loss	Low	Critical	Offline/HSM storage, multi-party backup, tested rotation root.
R-12	Overstated privacy claims	Med	High	Guarantee matrix, release claim checklist, legal/security review.
R-13	Version/API churn	High	Med	Pinned toolchains, quarterly review, compatibility fixtures and migration budget.

Appendix A. Technology baseline and version policy

Exact versions are pinned in version catalogs, lockfiles, rust-toolchain.toml, Foundry configuration and CI images. The table records the implementation baseline, not an evergreen promise. Every security update triggers compatibility tests before adoption.

Layer	Selected baseline	Version rule
Android	Kotlin 2.x, current stable Android Gradle Plugin, Jetpack Compose BOM	Pin exact build toolchain; target latest stable SDK at release.
Local DB	sqlcipher-android current maintained API (4.17.x baseline) plus AndroidX SQLite	Do not use deprecated android-database-sqlcipher API.
DM	org.signal:libsignal-client + org.signal:libsignal-android	Use official Signal artifact repository; pin same version for both.
MLS	OpenMLS 0.8.x baseline via reviewed Rust-to-Android bridge	RFC 9420 interoperability fixtures required.
PQC	ML-KEM only through reviewed protocol/library integration	Conform to FIPS 203; no checklist-only standalone KEM.
Media	AndroidX Media3/ExoPlayer	Use authenticated streaming data source; no plaintext disk cache.
Relay	Rust stable pinned, Axum/tonic, RocksDB, Noise implementation	Pin crate graph and run cargo audit/fuzz.
Contracts	Solidity 0.8.x compiler pinned, Foundry pinned	Invariant/fuzz and bytecode reproducibility required.
ZK	Noir/Nargo and proving backend pinned as one compatibility set	Beta/stability risk tracked; circuit and verifier key versioned.
L2	Base Sepolia for staging, Base mainnet for production USDC	Chain IDs, token addresses and confirmations configuration-signed.
Infrastructure	Docker/OCI, declarative deployment, independent relay failure domains	Immutable images, SBOM and staged rollout.

Appendix B. Repository structure

```
ghost/
|-- android/                # Android client
|   |-- app/
|   |-- identity/
|   |-- crypto-bridge/
|   |-- messaging/
|   |-- storage/
|   |-- sync/
|   |-- network/
|   |-- wallet/
|   |-- entitlement/
|-- relay/                  # Rust blind relay
|   |-- crates/api/
|   |-- crates/storage/
|   |-- crates/transport/
|   |-- crates/gossip/
|   |-- crates/prune/
|-- protocol/               # Protobuf and shared test vectors
|-- contracts/              # Solidity + Foundry
|-- circuits/               # Noir + verifier artifacts
|-- test-harness/           # E2E, load, chaos and fixtures
|-- infra/                  # Staging/production deployment
|-- docs/                   # ADR, threat model, runbooks, policies
|-- scripts/                # Reproducible build/release helpers
```

Appendix C. Production acceptance checklist

ID	Mandatory production condition
C-01	All P0 requirements have passing evidence linked to release commit.
C-02	All NFRs have recorded environment, workload, percentile and artifact version.
C-03	External mobile/protocol/relay/contract/circuit audit is retested; no open Critical/High.
C-04	Three production relays pass outage, partition, disk-pressure and rotation drills.
C-05	Identity recovery, DB migration and protocol-state crash recovery pass on supported Android versions.
C-06	Entitlement/referral flow reconciles and resists invalid, expired, replayed and self-referral cases.
C-07	Signed update rejects tamper and rollback; emergency key-rotation drill is complete.
C-08	SBOM, dependency locks, provenance and APK hash/signature are published or archived.
C-09	Incident runbooks, on-call, security contact, status communication and rollback are active.
C-10	Marketing, UI and legal disclosures match the guarantee matrix and enabled payment rails.
C-11	Go/No-Go is signed by product, technical, security, operations and financial owners.
C-12	Rollout plan uses 5/25/50/100 percent health gates with an explicit rollback threshold.

Appendix D. Normative references

- R1** BIP-39: Mnemonic code for generating deterministic keys
<https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>
- R2** Signal libsignal: official Java/Android artifacts and protocol implementation
<https://github.com/signalapp/libsignal>
- R3** RFC 9420: The Messaging Layer Security (MLS) Protocol
<https://www.rfc-editor.org/rfc/rfc9420.html>
- R4** NIST FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard
<https://doi.org/10.6028/NIST.FIPS.203>
- R5** Android Keystore system
<https://developer.android.com/privacy-and-security/keystore>
- R6** Android backup security recommendations
<https://developer.android.com/privacy-and-security/risks/backup-best-practices>
- R7** SQLCipher documentation and sqlcipher-android migration
<https://www.zetetic.net/sqlcipher/documentation/>
- R8** Noir documentation: public/private inputs and assert constraints
https://noir-lang.org/docs/language/data_types
- R9** Noise Protocol Framework
<https://noiseprotocol.org/noise.html>
- R10** RFC 5869: HKDF
<https://www.rfc-editor.org/rfc/rfc5869.html>
- R11** RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)
<https://www.rfc-editor.org/rfc/rfc8032.html>
- R12** RFC 7748: Elliptic Curves for Security (X25519)
<https://www.rfc-editor.org/rfc/rfc7748.html>

Appendix E. Change log

Version	Date	Summary
2.0	2026-08-20	Final baseline: corrected key/recovery model; scoped privacy guarantees; normative acceptance criteria; maintained protocol libraries; coherent relay/media limits; entitlement design; secure update; testing, roadmap, risks and production gates.
1.0	2026-08-19	Initial implementation-ready draft.

Final status

This document is the implementation baseline. Production authorization still requires completion of the gates, measured NFR evidence and an independent security audit with retest.